

Chapter 5

Replication

Keeping copies on multiple machines — and handling the changes that make it hard

Source: [claude-skills / ch05-replication.md](#) · Illustrated · Interview-ready

Martin Kleppmann · Part II: Distributed Data

Table of Contents

1. Core Idea
2. Framework: Single-Leader Replication
3. Replication Log Implementation Formats
4. Sync, Async & Semi-Synchronous
5. Bootstrapping a New Follower
6. Node Outages, Failover & Split Brain
7. Replication Lag & Consistency Guarantees
8. Multi-Leader Replication & Conflicts
9. Framework: Leaderless Quorum Replication
10. Sloppy Quorum & Hinted Handoff
11. Version Vectors, Siblings & Tombstones
12. Mental Models
13. Anti-Patterns
14. Worked Example: Quorum Reasoning
15. Problems Each Mechanism Solves
16. Connects To (Cross-Chapter)
17. Key Takeaways
18. Junior SWE Checkpoints

1. Core Idea

Replication — keeping copies of the same data on multiple machines — buys you:



Lower latency

Data near users geographically.



Higher availability

Survive node or datacenter failure.



Read throughput

Scale-out reads across replicas.

Replicating immutable data is trivial. All the difficulty is in handling changes. Three algorithms cover almost every distributed database: single-leader, multi-leader, and leaderless. Each trades write availability against consistency and conflict-handling complexity.

2. Framework: Single-Leader Replication

Also: master-slave, primary-replica, active/passive. **Default choice** — PostgreSQL, MySQL, MongoDB, Kafka.

When	Default choice; easy to reason about; no conflict resolution needed.
How	All writes go to one leader. Leader ships a replication log (change stream) to followers, which apply writes in the same order. Reads can hit any replica.
Why it works	Single node serializes all writes → one authoritative order.
Failure mode	Leader failover. Async replication: promoted follower may lack latest writes (discarded = durability violation). Split brain, failover storms.



Figure 2.1 — Single-leader: one write order, followers replay the log.

Real systems: PostgreSQL (streaming replication), MySQL (binlog), MongoDB (replica sets), Redis (`replicaof`), Kafka (partition leader + ISR followers).

3. Replication Log Implementation Formats

What goes in the log determines coupling, correctness traps, and whether you can feed CDC (Chapter 11).

Format	Ships	Breaks / Limits	Systems
Statement-based	Raw SQL statements	Breaks on <code>NOW()</code> , <code>RAND()</code> , ordering-sensitive statements, non-deterministic triggers	Legacy MySQL
WAL shipping	Storage-engine page bytes	Couples replicas to byte format → blocks zero-downtime version upgrades	Postgres physical replication
Logical / row-based	Row-level change records	Decoupled, version-tolerant, feeds CDC	MySQL row binlog, Postgres logical replication, MongoDB oplog
Trigger-based	App changelog table via triggers	Flexible but higher per-write overhead	Custom/legacy

Statement-based vs Logical log

Statement: `INSERT ... VALUES (NOW(), RAND())`
Follower evaluates at different time → diverges

Logical: `{row:42, ts:"2026-07-27", val:0.73}`
Exact resolved values — safe to replay anywhere

CDC pipeline (Ch 11)

Logical log → Debezium → Kafka → Redis cache / Elasticsearch index
22 Hotel Reservation (inventory cache) · 23 Email Service (async ES reindex)

Figure 3.1 — Logical logs decouple replicas and feed derived stores without dual-write.

4. Sync, Async & Semi-Synchronous

Mode	Behavior	Trade-off
Synchronous	Leader waits for follower ack before confirming write	Follower guaranteed up-to-date; one slow/dead follower blocks all writes
Asynchronous	Leader confirms immediately; followers catch up later	Fast; confirmed writes can be lost on leader failure
Semi-synchronous	One sync follower, rest async → data on ≥ 2 nodes	Pragmatic durability/availability compromise

📧 **Certified mail (sync) vs drop in mailbox (async).** Semi-sync = one signed receipt required, rest can catch up later. Kafka ISR ack modes (`ACK=all/1/0` , `replica.lag.max.messages`) tune this same durability-vs-latency knob per write (19 Message Queue).

All-followers-synchronous: any one node down halts all writes. Rarely used in production.

5. Bootstrapping a New Follower

Can't copy files off a live leader — you get a torn, inconsistent snapshot.

- 1 **Consistent snapshot** at a specific log position (`pg_basebackup` , MongoDB initial sync)
- 2 **Copy snapshot** to new node (can take hours)
- 3 **Request replication log** from that exact position
- 4 **Replay forward** to catch up

Risk: if snapshot + catch-up takes too long, leader may garbage-collect that log segment. Production systems retain segments long enough or parallelize transfer.

6. Node Outages, Failover & Split Brain

Follower down → easy

Leader keeps writing. Follower reconnects, requests log from last processed position, catches up.

Leader down → failover (where the bodies are buried)

1. Detect leader dead (timeout / heartbeat — usually 10–30s)
2. Promote most caught-up follower
3. Reconfigure all nodes and clients

GitHub 2012 — stale follower promoted

Failover promoted a stale MySQL replica. New leader missed recent auto-increment key assignments. Old leader rejoined; IDs reused → **new rows overwrote existing rows**. Private repository data leaked into public Redis search indexes keyed on those IDs.

Lesson: promote the most caught-up replica, or use consensus election (Ch 9). Auto-failover with aggressive timeouts under load → **failover storms** that make a struggling system worse. Some ops teams prefer manual failover for exactly these reasons.

Split brain

Two nodes both believe they're leader, both accept writes → data loss/corruption. **Fencing** forces old leader to step down (STONITH). Done badly, STONITH can shoot *both* nodes. Consensus-backed election (Raft/Paxos) eliminates split brain — not just heartbeat failover.

7. Replication Lag & Consistency Guarantees

"**Eventually consistent**" is deliberately unbounded — lag is sub-second normally, minutes under load. Design for: *"what if lag is an hour?"*

Guarantee	User expectation	Implementation
Read-your-writes (read-after-write)	Always see own submitted data	Read profile from leader; read from leader for 1 min after write; client tracks timestamp/LSN and only reads caught-up replicas. Cross-device is harder — metadata must be centralized; devices may route to different DCs.
Monotonic reads	Never see time go backwards across reads	Pin each user to one replica (hash of user ID)
Consistent prefix reads	Causally ordered writes seen in order	Mainly a sharding problem — independent partitions have no global write order. Connects to snapshot isolation (Ch 7).
Linearizability	Single-copy illusion	Requires consensus (Ch 9) — not automatic on followers

📱 **Social media post disappears on refresh** — read-your-writes violation. Write hit leader; refresh read lagging follower. Fix: route post-write reads to leader or track write position.

8. Multi-Leader Replication & Conflicts

Each datacenter (or device, or browser tab) has a leader; leaders exchange changes async.

Wins

Local write latency, DC-outage tolerance, offline operation (calendar sync), collaborative editing (15 Google Drive).

Cost

Write conflicts are inevitable and detection is asynchronous — too late to ask the user. Breaks auto-increment/triggers/constraints.

"Dangerous territory" — widely considered when retrofitted onto single-leader DBs. PostgreSQL BDR lacks causal ordering; Tungsten doesn't even detect conflicts. Read the docs, test the guarantees.

Conflict Handling (best → worst)

Strategy	How	Risk
Avoidance (best)	Route each record's writes to one home leader	Defeats multi-DC write benefit for that key
Last-write-wins (LWW)	Highest timestamp wins; discard rest	Silently drops acknowledged writes. Clock skew (Ch 8) drops even non-concurrent writes. Only safe for write-once immutable keys (Cassandra UUID-key recommendation). Cassandra's only conflict mode.
Merge / siblings	Keep both concurrent values; app merges	App must handle merge logic
User resolution	Surface conflict to user (CouchDB on-read handlers)	UX cost
CRDTs / OT	Automatic merge (research/production: Google Docs OT)	Domain-specific

9. Framework: Leaderless Quorum Replication

Dynamo-style: Cassandra, Riak, Voldemort, DynamoDB. **No failover needed at all.**

When	High availability and low write latency matter more than strict consistency.
How	n replicas. Write acked by w nodes, read queries r nodes. If $w + r > n$, read and write sets overlap $\rightarrow$ at least one read replica has latest value. Version numbers pick winner. Read repair + anti-entropy fix stragglers.
Why it works	Pigeonhole overlap — set intersection.
Failure mode	Overlap has real holes: sloppy quorums break it, concurrent writes have no defined winner, write that succeeded on $< w$ replicas isn't rolled back on nodes where it landed. Probability knob, not absolute guarantee. No read-your-writes or monotonic reads.

$$n=5, w=3, r=3 \rightarrow w+r=6 > 5 \checkmark$$

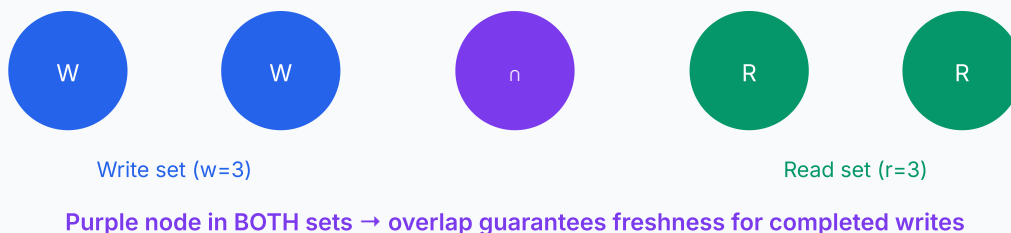


Figure 9.1 — Quorum overlap arithmetic.

Tolerance math: $w < n \rightarrow$ writes survive one node down; $r < n \rightarrow$ reads survive one down; $n=5, w=r=3 \rightarrow$ tolerates two down.

10. Sloppy Quorum & Hinted Handoff

When client can't reach the n "home" nodes, accept writes on any reachable w nodes — *"crash on the neighbor's couch."* Once partition heals, hand writes back home (hinted handoff).

Not a real quorum. Only a durability assurance. Reads may miss the value until handoff completes. The moment writes land outside the home set, the $w+r>n$ intersection — and the guarantee — **evaporates**.

Overlap arithmetic mental model: every quorum claim reduces to set intersection. Sloppy quorum, hinted handoff, partial failures → intersection dies. You kept durability, not consistency.

11. Version Vectors, Siblings & Tombstones

Concurrency is about knowledge, not wall-clock time.

A happens-before B iff B knew about / built on A. If neither knew about the other, they're concurrent — regardless of physical timing. Three cases only: $A \rightarrow B$, $B \rightarrow A$, or concurrent. Concurrent = conflict must be resolved.

Version vectors: one version counter per replica per key. Sent to clients on read, back on write. Server distinguishes overwrite (happens-before) from concurrent write (keep both as **siblings**).

```
Replica 1: {color: "blue", vv: [A:1]}
Replica 2: {color: "red", vv: [B:1]}
→ Concurrent siblings — app must merge, NOT "keep both" blindly for deletes
```

Tombstones — deletions are special

Physical delete + sibling merge without tombstones → **deleted items resurrect** (Amazon shopping cart bug). A delete must write a tombstone marker that wins over older live values during merge. GC tombstones after all replicas have seen them (`gc_grace_seconds` in Cassandra).

Without tombstone → resurrection

User deletes item X

Replica 3 misses delete

Merge siblings

Item X reappears in cart!

With tombstone: {key:X, value:TOMBSTONE, vv:[A:3]} beats stale live value

Figure 11.1 — Tombstones prevent resurrection on merge.

12. Mental Models

1. Knowledge, not time

Concurrency = neither writer knew about the other. Makes LWW's clock-skew failure obvious, not surprising.

2. Overlap arithmetic

Every quorum claim = set intersection. Sloppy quorum / partial failure → intersection evaporates.

3. Promise ledger

Async replication = DB confirmed writes it can still lose. Every anomaly (lost failover writes, stale reads, resurrected carts) is a confirmed promise quietly broken.

13. Anti-Patterns

Anti-pattern	Why it fails
Pretending async replication is synchronous	"Recipe for problems down the line"
LWW on mutable keys	Acknowledged writes vanish; clock skew makes it worse (Cassandra default!)
Auto-failover with aggressive timeouts under load	Failover storms. GitHub: stale promotion → ID reuse → data leak via Redis index
Fencing missing on failover	Split brain; bad STONITH shoots both nodes
Trusting multi-leader conflict detection blind	BDR lacks causal ordering; Tungsten doesn't detect conflicts
Merging siblings with deletions, no tombstones	Deleted items reappear (Amazon cart bug)
All-followers-synchronous	Any one node down halts all writes

14. Worked Example: Quorum Reasoning (n=3, w=2, r=2)

Happy path

```
User 1234 updates profile picture.  
Write fans out to all 3 replicas. Replica 3 is rebooting.  
Replicas 1 & 2 ack (version 7) → w=2 satisfied. Write succeeds.  
  
User 2345 reads: r=2 responses — replica 1 (v7) and replica 3 (v6).  
w + r = 4 > 3 → read set MUST include a written node.  
Version numbers: v7 wins. Client returns v7.  
Read repair: client pushes v7 back to replica 3.
```

Break it: sloppy quorum

```
Network partition cuts client from replicas 1-3.  
Sloppy quorum accepts write on two OUTSIDE nodes → w=2 "succeeded."  
Subsequent r=2 read of HOME nodes sees only v6.  
Overlap argument silently died. You kept durability, not consistency.
```

Edge case (not "node down during read"): A write the client considers **failed** (timeout before w acks) may have landed on 1-2 nodes and is **never rolled back**. Later read may return that ghost write — durability without client-visible consistency.

15. Problems Each Mechanism Solves

Mechanism	Problem it solves	Example
Single-leader + async	Simple default, ordered writes, cheap read scaling	Master→slave scaling, most CRUD backends
Semi-sync / ISR ack modes	Trade durability vs. latency per write	Message queue (Kafka ACK=all/1/0)
Leaderless quorum (w+r>n)	High write availability, no failover step	Dynamo-style KV store; multi-DC rate-limiter counters
Sloppy quorum + hinted handoff	Survive partition without blocking writes	KV store — breaks overlap guarantee
Version vectors + siblings	Detect concurrent writes instead of silently dropping	Google-Drive-style sync conflict surfaced to user
Multi-leader	Local-write latency across DCs / offline clients	Multi-DC document store, collaborative editing
Logical/row-based log → CDC	Keep derived store in sync without dual-write	Hotel reservation cache (Debezium→Redis), async ES reindex
Consensus-backed leader election	Eliminate split-brain entirely	Object storage placement, distributed wallet, matching engine

16. Connects To (Cross-Chapter)

Chapter	Connection	Interview soundbite
Ch 6 Partitioning	Replication assumed full dataset per node; sharding relaxes that. Consistent-prefix anomalies are fundamentally cross-partition.	"Each shard has its own leader + followers."
Ch 7 Transactions	Stronger guarantees than app workarounds; snapshot isolation for consistent prefixes.	"Lag means read-your-writes is an app concern, not automatic on followers."
Ch 8 Distributed Troubles	Unreliable clocks → LWW and timestamp ordering fail. Partitions → split brain.	"Never order events by wall clock."
Ch 9 Consensus	Leader election is consensus; linearizability vs quorums; $w+r>n$ timing edge cases break.	"Quorum $\neq$ linearizability. Need Raft for single-copy semantics."
Ch 11 Stream/CDC	Logical replication logs feed change data capture.	"One system of record + CDC, not dual-write."

17. Key Takeaways

1. **Single-leader is the default** for a reason: one write order, no conflicts. Its price is failover — lost writes, split brain, timeout tuning.
2. **Semi-synchronous** (one sync follower) is the pragmatic durability/availability compromise.
3. **Lag anomalies have names and fixes:** read-your-writes (leader reads / timestamp tracking), monotonic reads (replica pinning), consistent prefix (causal ordering per partition). Cite these precisely in interviews.
4. **$w + r > n$** gives probabilistic freshness, not linearizability — enumerate edge cases (sloppy quorum, concurrent writes, partial write failure) before claiming "quorums are consistent."
5. **Multi-leader/leaderless** buy write availability by deferring conflicts; someone must converge — LWW (lossy), merge/siblings + version vectors (safe but app merges), or CRDTs.
6. **Conflict avoidance** (one home leader per record) beats conflict resolution whenever routing allows.

18. Junior SWE Checkpoints

Q1: Why doesn't async replication guarantee durability?

Leader confirms write before followers receive it. Leader crash = write existed only on dead node. The promise ledger broke.

Q2: User posts, refreshes, post missing. What guarantee violated?

Read-your-writes. Read hit lagging follower. Fix: leader read after write, or track LSN/timestamp.

Q3: $n=3$, $w=2$, $r=2$ — what's the real stale-read edge case?

Not "one node down during read" — completed writes overlap ($w+r=4>3$).

Real cases: (1) Ghost write — client timeout before w acks, but write landed on 1–2 nodes, never rolled back. (2) Sloppy quorum — write on substitute nodes outside home set; $r=2$ read of home nodes sees stale v_6 despite $w=2$ "success."

Q4: Why can't statement-based replication use NOW()?

Follower evaluates NOW() at replay time → different timestamp → divergence. Logical log ships resolved value.

Q5: Deleted cart item reappears after sync. Root cause?

Missing tombstone. Lagging replica still had item; merge resurrected it. Amazon cart bug pattern.

Q6: How does replication connect to transactions (Ch 7)?

Single-leader gives ordered writes enabling snapshot isolation locally. Replication lag means committed write on leader may not be visible on follower — read-your-writes is app-level, not transactional guarantee across replicas.

Glossary

Term	Definition
Replication log	Ordered change stream from leader to followers
Statement-based log	Raw SQL — breaks on non-deterministic functions
WAL shipping	Page bytes — couples engine version
Logical/row-based log	Row changes — decoupled, feeds CDC
Tombstone	Delete marker preventing resurrection on merge
Version vector	Per-replica counters tracking causal knowledge
Siblings	Concurrent values kept alongside each other
Sloppy quorum	Write outside home set — breaks overlap math
Read repair	Push newer value to stale replica during read
Anti-entropy	Background diff-and-copy between replicas
Split brain	Two leaders accepting conflicting writes
CDC	Change Data Capture — tail logical log for derived stores (Ch 11)